

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики
Кафедра математичної інформатики

Курсова робота
за спеціальністю 122 Комп'ютерні науки
на тему:

**МОНОКУЛЯРНА ВІЗУАЛЬНА ОДОМЕТРІЯ
В РЕАЛЬНОМ ЧАСІ ДЛЯ БПЛА**

виконала студентка 3-го курсу

ОПП «Інформатика»

Уточкіна Яна Максимівна

(підпис)

Науковий керівник:

завідувач кафедри теоретичної кібернетики

доктор фізико-математичних наук

Терещенко Василь Миколайович

(підпис)

Засвідчую, що в цій роботі
немає запозичень з праць інших авторів
без відповідних посилань.

Студентка

(підпис)

РЕФЕРАТ

Обсяг роботи 21 сторінок, 2 ілюстрацій, 1 таблиці, 9 джерел посилань.

ВІЗУАЛЬНА ОДОМЕТРІЯ, КАЛІБРУВАННЯ КАМЕРИ, МОНОКУЛЯРНА КАМЕРА, ОПТИЧНИЙ ПОТІК, ВЕБ-ЗАСТОСУНОК, КОМП'ЮТЕРНИЙ ЗІР.

Об'єктом роботи є процес оцінювання траєкторії руху монокулярної камери за послідовністю зображень у режимі реального часу або з відеофайлу. Предметом роботи є методи детекції та відстеження ознак, відновлення відносної пози камери та програмна реалізація веб-системи візуальної одометрії.

Метою роботи є розробка та дослідження програмного комплексу монокулярної візуальної одометрії з веб-інтерфейсом для візуалізації тривимірної траєкторії.

Методи розроблення: аналіз літератури з візуальної одометрії, проектування клієнт–серверної архітектури, реалізація алгоритмів комп'ютерного зору (OpenCV), компонентна веб-розробка (Angular), експериментальна перевірка на наборах даних EuRoC MAV. Інструменти: Python, FastAPI, OpenCV, Angular, Plotly.js, Docker.

Результати роботи: спроектовано та реалізовано веб-систему з WebSocket-транспорт; розроблено модуль VO на базі Shi–Tomasi, Lucas–Kanade та суттєвої матриці з RANSAC; реалізовано калібрування камери за шаховою дошкою; отримано профіль камери з reprojection error близько 1,21 px на даних EuRoC.

Система може застосовуватися для навчальної демонстрації принципів візуальної одометрії, прототипування алгоритмів локалізації БПЛА та попередньої оцінки траєкторії за відео з бортової камери.

ЗМІСТ

Реферат	1
Зміст	2
Скорочення та умовні позначки	5
Вступ	6
1 Теоретичні основи монокулярної візуальної одометрії	8
1.1 Пін-hole модель камери	8
1.2 Калібрування камери	8
1.3 Детекція та відстеження ознак	10
1.4 Епіпольна геометрія та суттєва матриця	10
1.5 Неоднозначність масштабу в монокулярній VO	11
1.6 Огляд підходів ORB-SLAM та PTAM	11
2 Проєктування системи	12
2.1 Загальна архітектура	12
2.2 Режим роботи	12
2.3 Протокол обміну даними	13
2.4 Зберігання конфігурації	13
2.5 Стек технологій	13
3 Реалізація програмного комплексу	14
3.1 Модуль візуальної одометрії	14
3.1.1 Попередня обробка	14
3.1.2 Конвеєр обробки кадру	14
3.1.3 Стани трекінгу	15
3.2 Модуль калібрування	15
3.3 Frontend та розгортання	15

4	Експериментальні дослідження	17
4.1	Набір даних EuRoC MAV	17
4.2	Результати калібрування	17
4.3	Оцінювання VO	18
4.4	Обмеження експериментів	19
	Висновки	20
	Перелік джерел посилання	21

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАКИ

API — програмний інтерфейс застосунку (Application Programming Interface);
ATE — абсолютна похибка траєкторії (Absolute Trajectory Error);
БПЛА — безпілотний літальний апарат;
GNSS — глобальна навігаційна супутникова система;
IMU — інерціальний вимірювальний блок;
LK — метод Lucas–Kanade;
RPE — відносна похибка пози (Relative Pose Error);
SLAM — одночасна локалізація та побудова карти (Simultaneous Localization and Mapping);
VO — візуальна одометрія (Visual Odometry);
YAML — мова серіалізації даних YAML Ain't Markup Language;
 E — суттєва матриця;
 K — матриця внутрішніх параметрів камери.

ВСТУП

Оцінка сучасного стану об'єкта розроблення.

Визначення положення та орієнтації мобільного робота або безпілотного літального апарата є фундаментальною задачею автономної навігації. У середовищах, де сигнал GNSS недоступний або ненадійний, основним джерелом інформації про рух часто стає бортова відеокамера. Візуальна одометрія дозволяє оцінювати відносний рух камери між послідовними кадрами шляхом інтегрування зміщень та обертань у траєкторію [5]. Монокулярна VO використовує один відеопотік, що спрощує апаратну частину, але створює труднощі через неоднозначність масштабу та накопичення похибки. Сучасні SLAM-системи, зокрема ORB-SLAM [2] та PTAM [3], демонструють високу точність, однак їх повна реалізація є складною для навчальних та прототипних задач.

Актуальність роботи та підстави для її виконання.

Розробка веб-орієнтованої системи, яка поєднує алгоритми комп'ютерного зору на сервері та інтерактивну візуалізацію траєкторії в браузері, актуальна для навчального процесу, швидкого прототипування та попередньої оцінки руху БПЛА за відео. Наявність відкритих наборів даних, зокрема EuRoC MAV [1], дозволяє верифікувати калібрування камери та якість VO.

Мета й завдання роботи.

Метою курсової роботи є розробка та дослідження програмного комплексу монокулярної візуальної одометрії з веб-інтерфейсом, що працює в режимі реального часу або з попередньо записаного відео. Для досягнення мети поставлено такі завдання:

- проаналізувати теоретичні основи монокулярної VO та калібрування камери;
- спроектувати клієнт–серверну архітектуру системи;
- реалізувати модуль оцінювання руху на основі оптичного потоку та епіпольярної геометрії;
- реалізувати калібрування камери за зображеннями шахової дошки;

- забезпечити візуалізацію тривимірної траєкторії та керування профілями камери;
- провести експериментальну перевірку на наборах даних EuRoC MAV.

Об’єкт, методи й засоби розроблення.

Об’єктом дослідження є процес оцінювання траєкторії руху монокулярної камери за послідовністю зображень. Предметом — методи детекції ознак, відстеження, відновлення пози та програмна реалізація веб-системи VO. У роботі застосовано методи аналітичної геометрії, комп’ютерного зору (OpenCV), проєктування клієнт–серверних систем, компонентний підхід у веб-розробці (Angular), а також експериментальну верифікацію на еталонних наборах даних.

Можливі сфери застосування.

Створена система може використовуватися для демонстрації принципів VO в навчальному процесі, швидкого прототипування алгоритмів локалізації, попередньої оцінки траєкторії БПЛА та збереження параметрів калібрування камери у профілях для подальших сеансів VO.

Структура роботи.

Робота складається з вступу, чотирьох розділів, висновків, переліку джерел посилання та додатків. У розділі 1 викладено теоретичні основи; у розділі 2 — проєктування системи; у розділі 3 — реалізацію; у розділі 4 — експериментальні дослідження.

1

ТЕОРЕТИЧНІ ОСНОВИ МОНОКУЛЯРНОЇ ВІЗУАЛЬНОЇ ОДОМЕТРІЇ

1.1 Пін-hole модель камери

Камеру часто апроксимують центральною перспективною (pin-hole) моделлю. Точка світового простору $\mathbf{P} = (X, Y, Z)^T$ проєктується на зображення через матрицю внутрішніх параметрів K :

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \quad K = \begin{bmatrix} f_u & 0 & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}, \quad (1.1)$$

де f_u, f_v — фокусні відстані в пікселях; c_u, c_v — координати головного пункту; λ — масштабний множник глибини.

На рис. 1.1 схематично показано проєкцію точки світового простору на площину зображення та систему координат OpenCV.

Для точної геометрії враховують дисторсію об'єктива, яку компенсують функцією `undistort` у OpenCV [4].

1.2 Калібрування камери

Калібрування — визначення K та коефіцієнтів дисторсії за зображеннями відомого просторового шаблону. Найпоширеніший підхід — шахова дошка (метод Zhang, OpenCV `calibrateCamera`):

1. для кожного кадра знаходять внутрішні кути сітки;
2. уточнюють їх субпіксельно;
3. мінімізують reprojection error — RMS-відстань між спостереженими та проє-

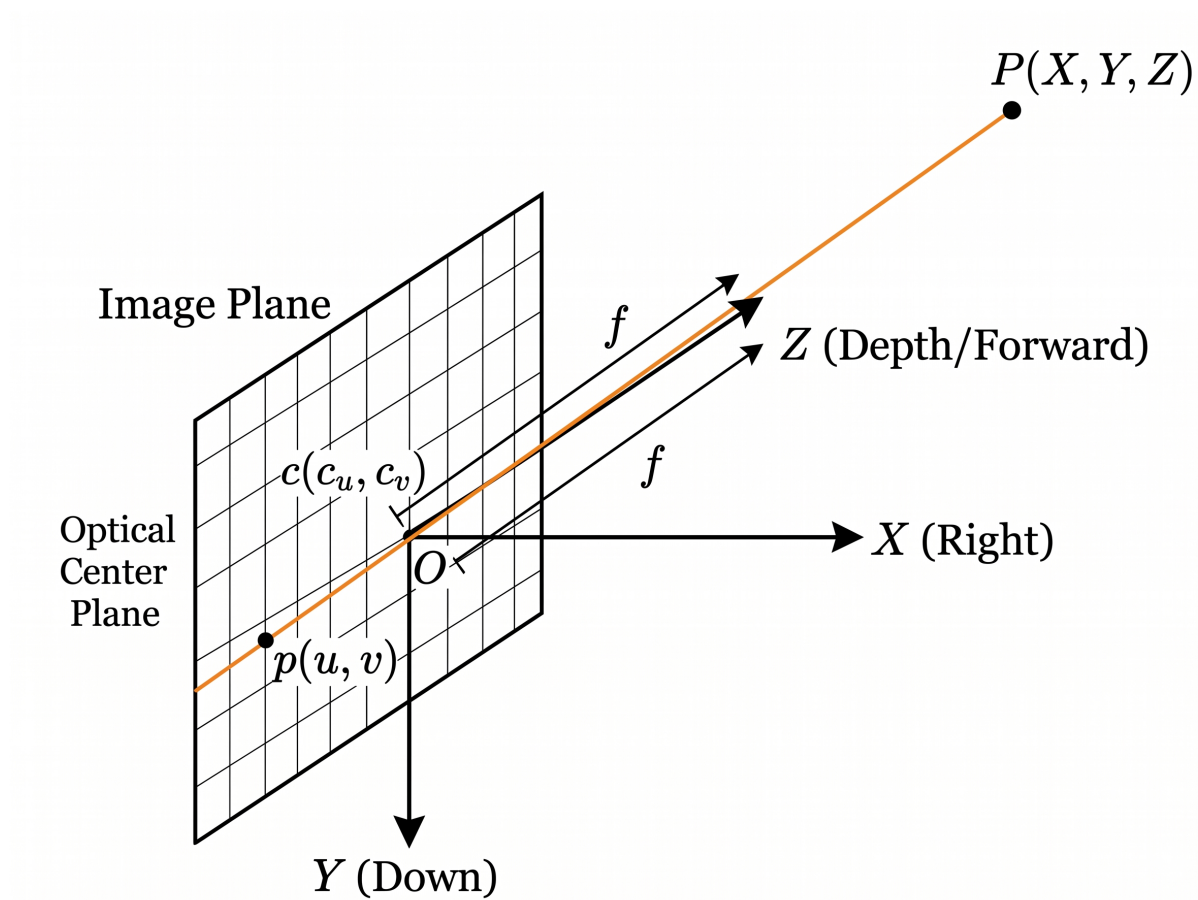


Рис. 1.1 – Pin-hole модель камери та система координат OpenCV

ктованими кутами.

Якість калібрування оцінюють за reprojection error: значення менше 0,5–1,0 px вважають прийнятними для VO.

1.3 Детекція та відстеження ознак

У розробленій системі застосовано:

- Shi–Tomasi (goodFeaturesToTrack) для вибору сильних кутів;
- Lucas–Kanade — пірамідальний оптичний потік calcOpticalFlowPyrLK;
- forward–backward consistency — відхилення треку, якщо зворотний потік не повертає точку в початкове положення.

Приклад відстеження ознак між двома послідовними кадрами наведено на рис. 1.2.

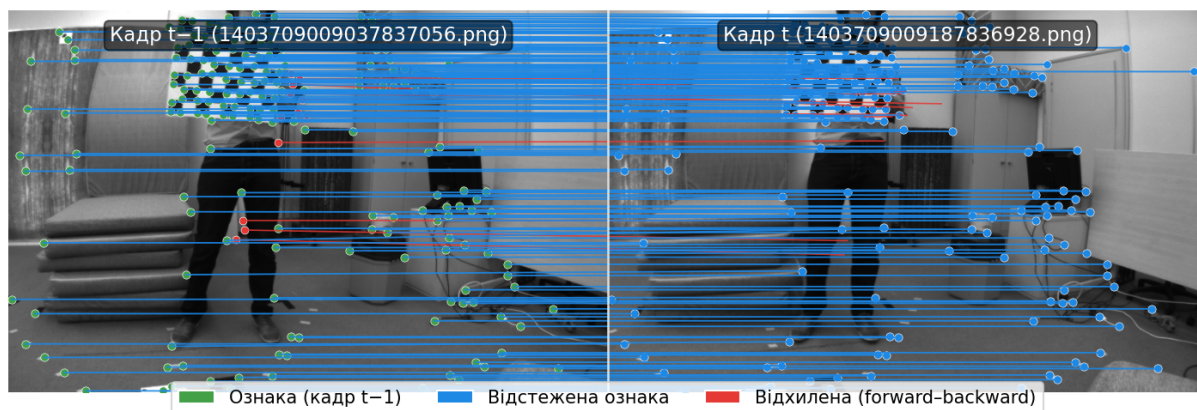


Рис. 1.2 – Відстеження ознак методом Lucas–Kanade між двома кадрами

1.4 Епіпольярна геометрія та суттєва матриця

Для двох кадрів з відомою калібрувкою K відносний рух описується суттєвою матрицею E [6]:

$$E = [\mathbf{t}]_{\times} R, \quad (1.2)$$

де R — відносне обертання, $\mathbf{t}$ — напрямок переносу (одиниця довжини), $[\mathbf{t}]_{\times}$ — кососиметрична матриця.

Матрицю E оцінюють методом RANSAC (`findEssentialMat`), після чого `recoverPose` відновлює R та $\mathbf{t}$ з перевіркою `cheirality`.

1.5 Неоднозначність масштабу в монокулярній VO

На відміну від стерео- або RGB-D-систем, одна камера не спостерігає абсолютну глибину без додаткових припущень. Вектор переносу $\mathbf{t}$ відновлюється лише з точністю до множника [5]. У даній роботі застосовано спрощений підхід: нормалізація кроку переносу та обмеження відносного масштабу між кадрами.

1.6 Огляд підходів ORB-SLAM та PTAM

Сучасні SLAM-системи (ORB-SLAM [2], PTAM [3]) поєднують надійне відстеження з дескрипторами, ключові кадри, локальну `bundle adjustment` та `loop closure`. Розроблена система реалізує полегшений VO-конвеєр, орієнтований на `real-time` обробку через `WebSocket`, без глобальної карти та замикання контурів.

ПРОЄКТУВАННЯ СИСТЕМИ

2.1 Загальна архітектура

Система побудована за архітектурою клієнт–сервер (рис. 2.1). Обчислювально складні операції комп’ютерного зору виконуються на Python-сервері; Angular-клієнт відповідає за захоплення відео, налаштування та візуалізацію.

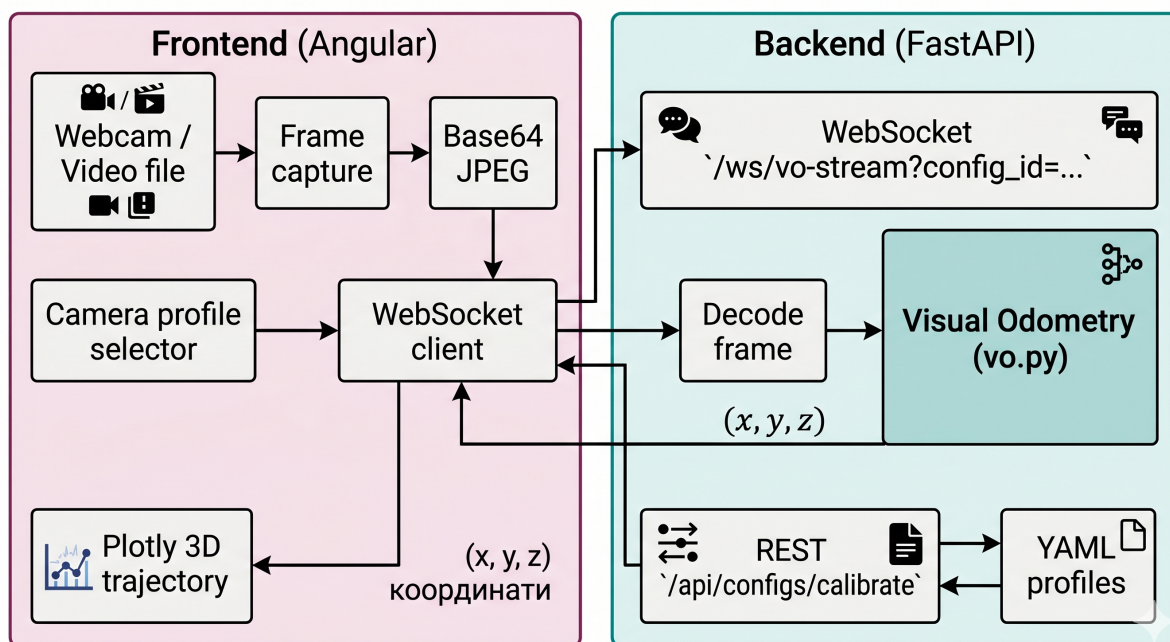


Рис. 2.1 – Логічна архітектура системи

2.2 Режими роботи

Підтримуються два режими: Stream — захоплення з веб-камери в реальному часі; From file — обробка завантаженого відеофайлу (наприклад, конвертованої послідовності EuRoC). Перед запуском VO користувач обирає профіль калібрування

камери.

2.3 Протокол обміну даними

З'єднання встановлюється через WebSocket: `ws://<host>:8000/ws/vo-stream?`

Цикл запит–відповідь: клієнт надсилає кадр (Base64 JPEG); сервер обробляє кадр у окремому потоці (`asyncio.to_thread`); сервер повертає координати та стан трекінгу; клієнт додає точку на 3D-графік і надсилає наступний кадр.

2.4 Зберігання конфігурації

Профілі камери зберігаються у форматі YAML у каталозі `vo-uav/storage/config`

Профіль містить внутрішні параметри f_u , f_v , c_u , c_v , коефіцієнти дисторсії, метадані калібрування та параметри VO.

2.5 Стек технологій

Стек технологій наведено в табл. 2.1.

Табл. 2.1 – Технологічний стек системи

Шар	Технології
Backend	Python 3.11+, FastAPI, OpenCV, NumPy, PyYAML
Frontend	Angular 21, TypeScript, Angular Material, Plotly.js
Транспорт	WebSocket, REST (калібрування, профілі)
Інфраструктура	Docker, Docker Compose

3

РЕАЛІЗАЦІЯ ПРОГРАМНОГО КОМПЛЕКСУ**3.1 Модуль візуальної одометрії**

Клас VisualOdometry (`vo-uav/src/vo.py`) реалізує інкрементальний VO-конвеєр. Система координат OpenCV: X — вправо, Y — вниз, Z — углиб сцени.

3.1.1 Попередня обробка

Виконується масштабування K до фактичної роздільності кадру (якщо вона відрізняється від калібрувальної) та опційна undistortion за коефіцієнтами з профілю.

3.1.2 Конвеєр обробки кадру

Для кожного кадру I_t :

1. відстеження ознак між I_{t-1} та I_t (LK + forward-backward);
2. оцінка відносної пози: основний шлях — E + RANSAC + recoverPose; резервний — estimateAffinePartial2D для плоских рухів;
3. оцінка масштабу кроку (triangulation depth ratio або median flow);
4. інтеграція пози; поповнення ознак; періодичне оновлення ключового кадру;
5. згладжування траєкторії (ЕМА + відсіювання стрибків).

Блок-схема конвеєра наведена на рис. 3.1.

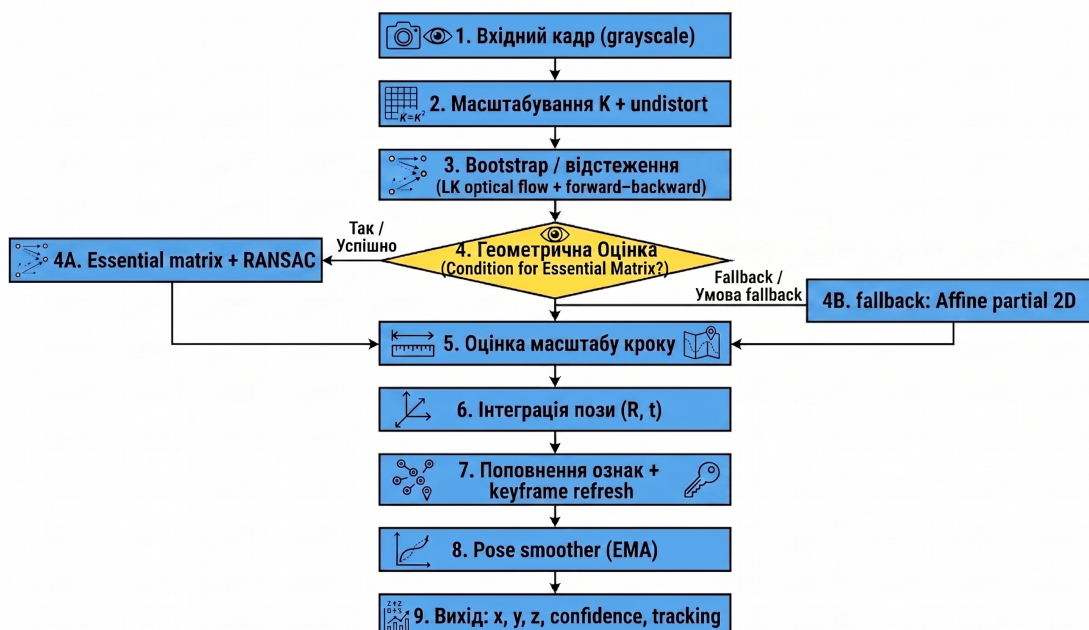


Рис. 3.1 – Блок-схема конвеєра візуальної одометрії

3.1.3 Стани трекінгу

API повертає поле `tracking` зі значеннями `initializing`, `ok`, `degraded`, `lost` для діагностики якості оцінки в інтерфейсі користувача.

3.2 Модуль калібрування

Файл `camera_calibration.py` реалізує побудову 3D-шаблону кутів шахової дошки, детекцію з `findChessboardCornersSB` та класичним `fallback`, автоматичний підбір розміру сітки (зокрема 6×7 для EuRoC `cam_checkerboard`), виклик `calibrateCamera` та збереження результату. REST-ендпоінт: `POST /api/configs/calibrate`.

3.3 Frontend та розгортання

Angular-додаток містить вкладки `Stream / From file`, елементи керування профілем камери, робочі простори VO з `Plotly scatter3d` та сервіс `WebSocket` з `watchdog`.

Головний інтерфейс показано на рис. 3.2. Розгортання здійснюється через `docker-compose.yml` (backend — порт 8000, frontend — порт 4200).

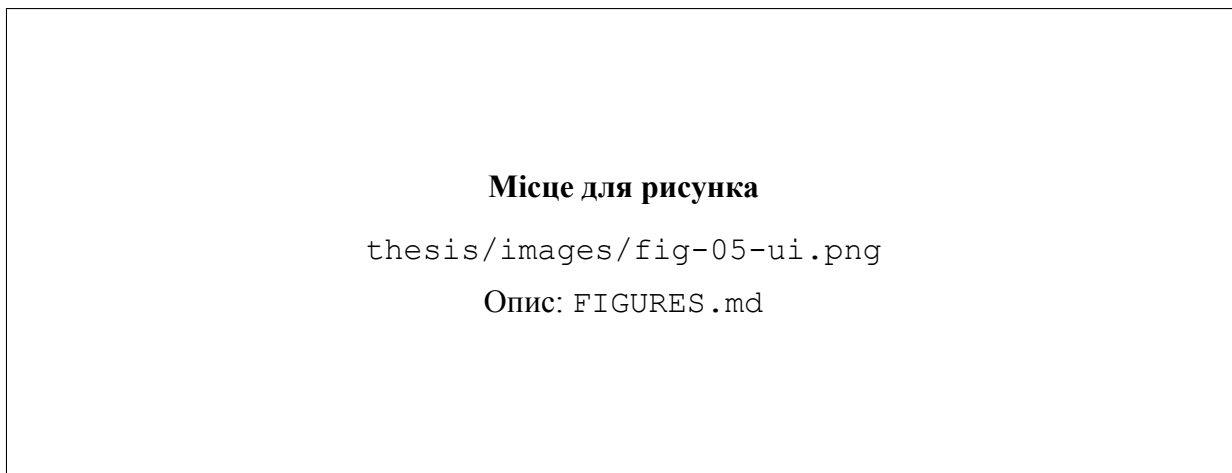


Рис. 3.2 – Головний інтерфейс системи (вкладка Stream)

ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ

4.1 Набір даних EuRoC MAV

Для тестування використано набір EuRoC MAV Dataset (ETH Zurich) [1] — стандартний бенчмарк для візуально-інерційної одометрії та SLAM. Послідовність `cam_checkerboard` містить кадри камери `cam0` (752×480 px, 20 Гц) з видимою шаховою дошкою. Для калібрування потрібна сітка 6×7 внутрішніх кутів. З 2376 кадрів дошку успішно детектовано на 1740.

4.2 Результати калібрування

Параметри профілю `dataset.yaml`, отриманого за 11 зображеннями з EuRoC, наведено в табл. 4.1.

Табл. 4.1 – Параметри калібрування (профіль Dataset)

Параметр	Значення
f_u	397,88 px
f_v	394,66 px
c_u	360,80 px
c_v	271,66 px
Reprojection error	1,21 px
Роздільність	752×480 px
Внутрішні кути	6×7
Розмір клітини	38 mm

Reprojection error близько 1,2 px — прийнятний рівень; для підвищення точності VO рекомендується більше різноманітних ракурсів. Приклад детекції кутів шахової дошки наведено на рис. 4.1.



Рис. 4.1 – Детекція внутрішніх кутів шахової дошки 6×7 (EuRoC cam_checkerboard)

4.3 Оцінювання VO

Якісна оцінка включає аналіз форми траєкторії на Plotly (рис. 4.2), полів confidence та tracking, поведінку при нерухомій камері та порівняння профілів EuRoC default і власної калібровки.



Рис. 4.2 – Тривимірна траєкторія, побудована системою VO (Plotly scatter3d)

Для кількісної оцінки на польових послідовностях EuRoC (наприклад, МН_01_ea

доступна ground truth. Рекомендований алгоритм: експорт траєкторії у формат TUM; Sim(3)-вирівнювання з ground truth; обчислення ATE та RPE за допомогою бібліотеки evo [7]. У поточній версії репозиторію автоматичний експорт траєкторії не реалізовано.

4.4 Обмеження експериментів

Монокулярна VO без loop closure накопичує drift; масштаб траєкторії умовний; webcam-сценарії часто активують affine-fallback; затримка мережі WebSocket впливає на частоту кадрів у режимі Stream.

ВИСНОВКИ

У курсовій роботі спроектовано та реалізовано веб-систему монокулярної візуальної одометрії з клієнт–серверною архітектурою, WebSocket-транспортом і 3D-візуалізацією траєкторії.

Розроблено VO-модуль на базі Shi–Tomasi, Lucas–Kanade, суттєвої матриці з RANSAC, резервного affine-методу та post-processing згладжування. Реалізовано калібрування камери за шаховою дошкою з автопідбором розміру сітки та збереженням профілів YAML.

На даних EuRoC `cam_checkerboard` отримано профіль з reprojection error близько 1,21 px; підтверджено коректність детекції дошки 6×7 .

Система придатна для навчальної демонстрації VO та прототипування. Перспективи розвитку: експорт траєкторії та порівняння з ground truth (evo), local bundle adjustment, ORB-дескриптори, fusion з IMU, loop closure.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Burri M., Nikolic J., Gohl P. [та ін.]. The EuRoC micro aerial vehicle datasets. International Journal of Robotics Research. 2016.
2. Mur-Artal R., Montiel J., Tardós J. D. ORB-SLAM: a versatile and accurate monocular SLAM system. IEEE Transactions on Robotics. 2015.
3. Klein G., Murray D. Parallel tracking and mapping for small AR workspaces. ISMAR. 2007.
4. Camera calibration with OpenCV // OpenCV Documentation. URL: https://docs.opencv.org/4.x/d4/d94/tutorial_camera_calibration.html (дата звернення: 29.05.2026).
5. Scaramuzza D., Fraundorfer F. Visual Odometry: Part I — The First 30 Years and Fundamentals. IEEE Robotics & Automation Magazine. 2011.
6. Hartley R., Zisserman A. Multiple View Geometry in Computer Vision. Cambridge University Press, 2004. 655 с.
7. evo: Python package for the evaluation of odometry and SLAM // GitHub. URL: <https://github.com/MichaelGrupp/evo> (дата звернення: 29.05.2026).
8. FastAPI Documentation. URL: <https://fastapi.tiangolo.com/> (дата звернення: 29.05.2026).
9. Angular Documentation. URL: <https://angular.dev/> (дата звернення: 29.05.2026).